

Problem Statement

TBX — BVP Tech Catalyst Hackathon

Problem Statement Title: Build a Finance Assistant That Actually Understands You

1. Background

- Finance teams and business users rely on dashboards, static reports, and exports to answer routine questions about their own financial data.
- Getting a simple answer, such as what was spent on a vendor last month, or which transactions are still unreconciled, usually means finding the right report, learning its terminology, or waiting on someone in finance ops to run a query.
- This slows down everyday decisions and pulls finance teams away from higher value work, since they end up fielding the same lookup questions on repeat.
- A conversational assistant that understands plain language and answers directly from real data would let anyone get a trustworthy answer in seconds, without opening a dashboard.
- The stakes are higher here than in most chatbot use cases. A wrong or invented number in a finance context is not a minor bug, it is a liability that can undermine reconciliation, audits, and trust in the numbers.

2. Challenge

Build a conversational AI assistant that lets users ask questions about their financial data in plain language and get clear, accurate answers back. Examples:

- How much did we spend on vendor payouts last month?
- Which transactions are still unreconciled?

The assistant should answer instantly, without the user touching a dashboard. Grounding is the crux of this challenge: every answer must come from real data, never from the model filling in a plausible-sounding number.

3. Requirements

Must Have

- Natural language query handling: accept free-form questions through a chat interface and correctly interpret intent, filters, and date ranges.
- Grounded retrieval: every answer must be produced by querying the provided financial dataset (transactions, vendor payouts, reconciliation status), not from the model's own assumptions.
- Accurate computation: filter, group, and aggregate the data correctly (by vendor, date range, status, category) before handing results to the language model, so the model explains a computed result rather than calculating one itself.
- Verifiable answers: every response should pair a plain language answer with the underlying records or a breakdown table, so the user can check it.
- Hallucination guardrails: when the data needed to answer a question doesn't exist, or the question is ambiguous, the assistant should say so clearly rather than invent a figure.
- Lightweight model constraint: see Section 8, Assumptions and Constraints. This is a scored requirement, not a suggestion.

- Multi-turn conversation: follow-up questions, like asking how that compares to the month before, should work without the user repeating context.
- Explainability: the assistant should be able to show what data it pulled and how it arrived at an answer, not just state the answer, so a user can trace the reasoning back to the source records.

Good to Have

- Export of the underlying breakdown to CSV or Excel.

Bonus

- Confidence signalling: the assistant flags when it is less certain about an answer instead of stating everything with equal confidence.
- A short note on model choice: which lightweight model was used, why, and what accuracy looked like against a sample question set.
- Simple anomaly callouts, for example flagging a vendor payout that looks unusually large compared to history while answering the original question.

Out of Scope

- Live integration with real banking or ERP systems
- Multi-tenant security, user roles, or production-grade authentication
- Support for every possible financial question. A well-scoped subset (spend, payouts, reconciliation) is enough

4. Resources Provided

- We will share a starter dataset covering transactions, vendor payouts, reconciliation status, chart of accounts, vendor list, and a data dictionary before the hackathon.
- A sample chart of accounts and vendor list to give the data structure
- A short guidance note on how the model-efficiency constraint will be evaluated
- We will provide each team with equal, capped API credits so access is not a deterrent. Free or locally hosted open-source models are also allowed.

5. Evaluation Criteria

- Accuracy & grounding 30%
- Model efficiency 20%
- Natural-language understanding 15%
- Functionality 15%
- User experience 10%
- Presentation 5%
- Business impact 5%

6. Submission Requirements

- Working prototype (chat interface plus backend)

- Architecture diagram
- README with setup instructions
- Sample questions and the corresponding answers produced by the assistant
- Presentation deck covering the problem, approach, model choice rationale, and demo flow